

Project: Smart Attendance System

Expected Effort (approximate): 40 hours per person

Due Dates: Week 13

Weightage: 30% of final grade

Project Review	Marks
Code - The code is modularized with multiple classes and methods. - The code is clean. - The code is designed and layered to known design principles. For example, Single Responsibility Principle (SRP). - The configuration parameters are externalized. https://reflectoring.io/externalize-configuration - Adherence to Java coding convention. http://www.oracle.com/technetwork/java/codeconventions-150003.pdf - The code is appropriately documented. - The variable, class, and method names are meaningful. - The code is indented using tabs or spaces consistently.	7
Application - Usability of your application, Core features complete, tested end-to-end. - It can be a GUI or web application. <i>The main consideration is usability not aesthetics (i.e. have a pretty UI).</i> - UI design to map fields - Nielsen's 10 heuristic evaluation https://www.nngroup.com/articles/ten-usability-heuristics/ - Submission - source code + readme file + presentation slides	16
Presentation - Time allocation: 10-12 minutes for presentation & demonstration. 5 minutes for Q & A. - Presentation Content: - State your team's object-oriented design (system architecture). - List all the open-source libraries used. If you evaluated a few, state why you picked that particular library. - List the algorithms explored and explain which one is effective any why? What are the advantages and disadvantages of different algorithms. - Demonstrate the features of your application. - Presentation is well-rehearsed. - Slides are clean and visible. - There is no unnecessary switching of laptops/applications. - The transition is smooth.	7
Penalty - Up to 10% penalty of your total marks if the application has any usability issues (include input validations) - Up to 50% penalty of your total marks if any of your Java source files does not compile or run.	

Project Rubrics

	Evaluation Scale (10 – 30)			
	10 – 14	15 - 19	20 - 24	25 - 30
Code	There was no attempt to modularize the code with multiple classes and methods.	There was some attempt to modularize the code. A lot of repetitive code.	Most of the code were modularized with the exception of a few ‘god’ classes.	Applied good object modularization, methods perform one function only.
	There are multiple coding styles within the program. It does not adheres to the Java coding convention. There was no use of packages.	The coding style was consistent, but it does not adheres to the Java coding convention. There is Java packages but the intent of each package wasn’t clear.	Most of the code follows Java coding convention with a few exceptions. Clear use of packages.	The code follows Java coding convention consistently. Clear use of packages.
	No obvious attempt to design the solution. It’s a brute-forced solution.	Some attempt to do design. There is entity classes found.	Good attempt on doing object design. There was also the idea of separating the model from the view.	The design was clean. reader can easily figure out the intent of each class and the relationships between classes. It’s a piece of art!
Application	The application is straightforward and is handled with an if-else statement. On top of that, it’s buggy.	The application is straightforward and is handled with an if-else statement, or the application has some complex situation but has kinks all over.	The application has some complex scenarios and are reasonably handled. The app is mostly intuitive with some kinks.	Application has a lot of complex scenarios, and they are well-handled (e.g. lots of conditions are considered and handled) with versatile functions.
	The solution does not work or has lots of bugs (e.g. stack trace, crashes).	The solution somewhat works but has usability issues or occasional bugs.	The solution works with some usability or occasional bugs.	The app is intuitive (e.g. all prompts, alerts, statuses, etc. are well placed). The solution works perfectly.
	The application UI is puzzling and/or hard to navigate.	The application UI is decent with some usability issues or occasional bugs.	The application UI is intuitive and user-friendly, with minimal usability issues	The application UI is exceptional, polished, and highly user-centric.
Presentation	The presentation missed the points required.	The presentation covers 50% of the main points but need not elaborate well.	The presentation covers most or all the main points and has not elaborated all the points fully.	The presentation covers all the main points and elaborates well.
	The presentation deck was unclear, messy and with bad contrast or small fonts.	The presentation deck mostly decent with occasional flaws (e.g. font, contrast).	The presentation deck is decent. The team used it effectively to explain their project.	The presentation deck was well-put together. Diagrams are clear and illustrate the design.
	The presentation is not well-paced, and is hard to follow.	The presentation has some pacing issues and/or is frequently hard to follow.	The presentation was mostly well-paced. However, the team needs to switch laptops or restart apps.	The presentation was well-paced.
	The team has not prepared or rehearsed on the demo.	The demo flow was mostly too fast or confusing. There was insufficient time allocated for the demo.	The demo flow was decent with an occasional hiccups.	The demo showed the features well.
	Q&A: The team struggles to provide accurate answers, and responses lack consistency. For example: 1. Many questions are not	Q&A: The team can answer most questions adequately, though some responses lack depth or clarity. For example: 1. Most questions are answered correctly, but	Q&A: The team answers nearly all questions accurately, demonstrating strong knowledge and consistency. For example: 1. Responses are thorough,	Q&A: The team provides exceptional responses, consistently delivering clear, comprehensive, and prompt answers. For example: 1. Answers are

	answered well or accurately. 2. Team members often provide differing answers to the same question, causing confusion. 3. Responses are frequently unclear, incomplete, or require significant follow-up.	explanations may be basic or leave some room for improvement. 2. Team members are mostly consistent, though a few answers may vary slightly. 3. There may be a few complex questions that the team cannot fully address.	well-explained, and require minimal follow-up. 2. Team members provide consistent answers to similar questions, showing alignment in understanding. 3. Only occasional, less-common questions might require slight clarification.	highly accurate, detailed, and address all aspects of each question with professionalism. 2. Team members are fully aligned, delivering consistent and coherent responses across all inquiries. 3. The team proactively addresses potential follow-up questions, ensuring a smooth, efficient Q&A experience.
--	--	--	---	---

Peer Evaluation

- If you have any teaming issues, do highlight to your instructors early and not wait until the peer evaluation.
- As a team, you will be given a single grade for the project. However, your individual grade for the project can be higher or much lower than the team grade based on peers' evaluation (and instructors' evaluation based on the team's interview and observation across the semester).
- We will severely penalize free-loaders, as well as team members who decide to complete the project alone.
- Guidelines
 - Please be frank when evaluating your peers and do the evaluation seriously.
 - You **must use the comments** field to explain why you think a particular individual deserves the score. Note that the system will not force you to add comments, but we require it. You need to input comments about every member in your team for every metric. Be frank and honest with your comments.
 - Your team peer evaluation score will not be compared against other teams' scores. Having a lower team score than another team will not affect your final grade.
- You are expected to evaluate yourself and your team members based on 4 criteria on a scale of 1 – 5.
 - 1. Commitment 2. Working with others 3. Quantity of work 4. Quality of work

	Peer Evaluation Scale (1 – 5)			
	1 – 2	2-3	3-4	4-5
Commitment	Often missed meetings (>75% of total meetings).	Missed one or two meetings without reasons and failed to inform members about absence prior to the meeting.	Missed one to two meetings with valid reasons and informed the team about absence prior to the meeting.	Turned up consistently for meetings. Initiated and set agenda for meetings.
	Frequently late for meetings by more than 30 minutes or left early after a short duration of time meetings.	Frequently late by 5-10 minutes and occasionally late for more than 30 minutes. Occasionally left early.	On time for meetings. Occasionally late for meetings by 5-10 minutes. Stayed for the whole duration of meetings.	Never late for meetings. Stayed for the whole duration of meetings.
	Little or no contribution to discussions and activities.	Some contributions to discussions and activities.	Consistently contributed to all discussions and activities.	Led the team in discussions and activities. Willing to follow when others took initiative to lead.
Working with others	Unwilling to interact with others. Inflexible and negative to new ideas suggested.	Usually helpful and receptive towards new ideas suggested by others. At times, insisted on his own way of doing things or implemented ideas without the team's consensus.	Helpful and receptive towards new ideas suggested by others. A pleasure to work with.	Expressed ideas clearly, honestly and with respect for others and their work. Willing to listen without arguing every point.
	Never bothered to contribute any ideas to the team.	Contributed some good ideas to the team.	Constantly suggested new ideas to the team. Not all the ideas were applicable though.	Consistently suggested new ideas or was able to build on ideas suggested by others and all of them were good or creative.
Quantity of work	Did less than their fair share of load.	Contributed a fair share of the team's load. However, did more of a specific type of task only (e.g. coding)	Contributed a fair share of the team's load in all aspects.	Contributed a fair share of the team's load in all aspects. Enthusiastically took on responsibilities in various tasks areas.
Quality of work	Work often had flaws and omissions that needed fixing.	Acceptable quality of work that needed frequent minor fixes or the occasional rework.	Good quality of work with occasional minor fixes.	High quality of work that went well beyond expectations.

1. Project's Background

Manual attendance tracking in educational or professional settings is time-consuming, error-prone, and vulnerable to issues like proxy attendance. A vision-based smart attendance system automates this process by detecting and recognizing faces from a webcam feed or uploaded media, marking attendance in real time, and storing records securely. This project provides hands-on experience in Java, emphasizing Object-Oriented Programming (OOP) to design modular, maintainable code, alongside GUI development, database integration, and computer vision techniques using OpenCV/JavaCV.

2. Learning Objectives

By completing this project, students will:

1. Design and implement a web application (e.g. using PHP) OR Java desktop application (e.g. using Swing or JavaFX) or any kind of application you are familiar with, applying OOP principles like encapsulation, inheritance, polymorphism, and abstraction.
2. Create modular class hierarchies and use design patterns to manage complex components like face recognition and attendance marking.
3. Integrate OpenCV/JavaCV for live video capture, face detection, and recognition, encapsulating these in well-defined classes.
4. Model a robust data structure with persistent storage using SQLite, leveraging abstraction for database operations.
5. Develop intuitive user interfaces adhering to usability heuristics, with event-driven interactions reflecting OOP's benefits.
6. Follow Java coding conventions, modular programming, and comprehensive documentation to produce maintainable code.

3. Scope

The system will:

1. Enroll students with facial data for recognition, using encapsulated data models.
2. Manage attendance sessions with real-time or manual marking, designed with polymorphic behaviors.
3. Detect and recognize faces to automate attendance, abstracting recognition algorithms.
4. Generate and export detailed reports, using composition for flexible formats.
5. Emphasize OOP to ensure modularity, extensibility, and clarity in code structure.

4. Functional Requirements

4.1. Core

1. **Student Enrollment & Management**
 - **Functionality:** Create, edit, and delete student profiles with fields: Student ID (unique alphanumeric, e.g., S12345), Name, Class/Group (e.g., CS102), and optional Email, Phone. Capture or import 10–20 face images per student via webcam or files (JPEG/PNG). Validate images (e.g., face detected, sufficient quality) before storing.

- **OOP Design Advice:**
 - Define a Student class encapsulating private fields (ID, name, facial data) with getters/setters, extending an abstract Entity class for shared CRUD logic.
 - Use a StudentManager class to handle enrollment operations, applying the Repository pattern for database interactions.
 - Store facial data (images or embeddings) in a separate FaceData class, linked to Student via composition.
- **Implementation:**
 - Webcam capture: User clicks “Capture” in GUI, system takes images at 1-second intervals, using OpenCV’s HaarCascadeClassifier for face detection OR any non-OpenCv face detection method.
 - Preprocess images: Convert to grayscale, normalize lighting (histogram equalization), crop to a predefined size if necessary, OR any other process which can benefit the performance.
 - Display: Show a searchable table of students with thumbnail previews.

2. Attendance Session Management

- **Functionality:** Create sessions with Course Name, Date (auto-set to today), Start/End Time, Location (optional), and a roster of expected students selected from enrolled list. Open/close sessions; prevent deletion of active sessions.
- **OOP Design Advice:**
 - Model as a Session class with private fields (name, date, roster), extending Entity.
 - Use a SessionManager class to manage session lifecycle, implementing the Facade pattern to simplify interactions.
 - Roster as a List<Student> with a Roster class for validation (e.g., no duplicates).
- **Implementation:**
 - GUI: Form for session creation, table for roster management (columns: Name, ID, Status, Timestamp, Method).
 - Statuses: Pending (default), Present, Absent, Late; updated in real-time during sessions.

3. Face Detection & Recognition

- **Functionality:** Capture live webcam feed, detect faces, preprocess (e.g. grayscale, normalize, resize), and recognize against enrolled students. Display bounding boxes, Name/ID, and Confidence Score (e.g., 85%) on video feed.
- **OOP Design Advice:**
 - Abstract Recognizer class with a recognize() method; concrete subclasses Recognizer_alg1, Recognizer_alg2 for polymorphic algorithm selection. There are many face recognition algorithms that you can use, you can use the opencv basic methods like histogram matching (e.g. the demo codes) or template matching, or you can use advanced deep learning based method (<https://pyimagesearch.com/2018/02/26/face-detection-with-opencv-and-deep-learning/>).

- FaceDetector class encapsulates OpenCV/JavaCV's HaarCascadeClassifier or Deep learning based detector (e.g. YOLO) or any face detection method you prefer, exposing a simple detectFaces() method.
- Use Strategy pattern for preprocessing pipeline (e.g., GrayscaleProcessor, Normalizer as interchangeable steps).
- **Implementation:**
 - Detection: Process frames at ≥ 8 FPS (frames per second), draw rectangles around faces.
 - Recognition: Compare against stored embeddings; accept matches above configurable threshold (e.g., 70%).
 - Feedback: Overlay results on GUI; log unrecognized faces with optional user prompts.

4. Marking Attendance

- **Functionality:**
 - **Automatic Marking:**
 - On recognition (confidence > threshold), verify student is in roster and not marked.
 - Mark as "Present" with timestamp and "Auto" method; if after "Late Threshold" (e.g., 15 mins), mark as "Late".
 - Prevent duplicates using a cooldown timer (10–30 seconds, configurable); update "Last Seen" instead.
 - Handle multiple faces sequentially, prioritizing higher confidence.
 - **Manual Marking:**
 - Interactive roster table with dropdowns (Present/Absent/Late) and optional notes (e.g., "Medical Leave").
 - Manual changes override auto-marks, logged with "Manual" method.
 - **Edge Cases:**
 - Low confidence (50–70%): Prompt user via pop-up (e.g., "Confirm: Is this John Lim?").
 - Session close: Auto-mark remaining roster as "Absent".
 - Any kinds of cases that you can come up.
- **OOP Design Advice:**
 - AttendanceMarker interface with markAttendance() method; concrete classes AutoMarker and ManualMarker for polymorphic marking.
 - Use Observer pattern to update GUI roster table on marking events.
 - AttendanceRecord class encapsulates Student, Session, Status, Timestamp, and Method.

5. Reporting & Export

- **Functionality:** Display session summaries (e.g., Present: 80%) in tables or charts. Export as CSV, XLSX, or PDF with Student ID, Name, Status, Timestamp, Confidence, Method, Notes.
- **OOP Design Advice:**

- ReportGenerator interface with generate() method; subclasses CSVReport, PDFReport for polymorphic output.
- Use Builder pattern for flexible report customization (e.g., include/exclude fields).
- **Implementation:**
 - GUI: Filterable report view; export button with file chooser.
 - Optional: Email reports using JavaMail API.

6. GUI & Usability

- **Functionality:** Sidebar/menu with tabs: Students, Sessions, Live Recognition, Reports, Settings. Live view shows webcam with overlays, camera controls, and exception messages (e.g., “No Camera”).
- **OOP Design Advice:**
 - MainView class composes subviews (StudentView, SessionView).
 - Use Observer pattern for real-time updates (e.g., roster changes).
- **Implementation:**
 - Accessibility: Keyboard navigation, high-contrast UI, tooltips.
 - Responsiveness: No freezes during recognition (use background threads).

7. Configuration & Logging

- **Functionality:** Settings for camera index, recognition threshold, timers, database path. Log events (e.g., “Enrolled Student S12345”) to attendance.log.
- **OOP Design Advice:**
 - Configuration class with externalized properties (e.g., properties file).
 - Logger Facade for Java Logging API, encapsulating log levels and formats.
- **Implementation:**
 - GUI Settings tab for user adjustments; save changes persistently.

4.2. Bonus Features (extra credits)

- Multi-face tracking (Composite pattern for grouped faces).
- Liveness detection (e.g., blink prompts, using LivenessChecker class).
- Mask-aware recognition (MaskedRecognizer extends Recognizer).
- Auto-session policies (Decorator pattern for adding rules).
- Batch roster import (Importer class with CSV parsing).
- Analytics dashboard (JFreeChart integration, ChartFactory).
- Cloud sync (Firebase, with CloudConnector adapter).
- Persistence: Store data in SQLite (tables: Students, Sessions, Attendance, FaceData). Implement CRUD with JDBC, ensuring transactions and backups. Store facial data as blobs (Binary Large Objects) or file paths; validate integrity on load.
-

Detailed Attendance Flow

1. Before Session:

- User navigates to Sessions tab, creates session via form (e.g., "CS102, 2025-08-22, 10:00-11:00"), selects roster.
- System validates inputs, saves to database with SessionManager, sets roster statuses to "Pending".

2. During Session:

- Open session: Start webcam in Live Recognition tab via CameraController.
- Loop: FaceDetector detects faces → Recognizer matches → AutoMarker checks roster/cooldown → Marks attendance → MainView updates table/overlays.
- Low confidence: MainView prompts user for confirmation.
- Manual override: User edits table in SessionView; ManualMarker saves changes.
- Feedback: Show counters (e.g., "Present: 15/20") and alerts (e.g., "Late: Jane Doe").

3. After Session:

- Close session: SessionManager stops feed, marks absentees, finalizes records.
- Reports tab: View/export via ReportGenerator; optional email via EmailSender.
- Archive: Sessions read-only; allow reopening with AuditLogger.

Non-Functional Requirements

- **Performance:** ≥8 FPS on mid-range hardware (Intel i5, 8GB RAM); recognition <300ms/face.
- **Accuracy:** ≥70% recognition rate under good lighting
- **Usability:** Intuitive UI, ≤5 clicks for key actions, responsive controls.
- **Security:** basic login for multi-user access.

OpenCV Installation

Windows:

Step 1: download opencv from: <https://opencv.org/releases/>

If your jdk version is 21 or below, please choose OpenCV – 4.8.0

Install it to somewhere on your computer, for example, I installed in D:\opencv on my computer.

Step 2: Go to the location where you installed your opencv, and for windows, copy the file:

D:\opencv\build\java\x64\opencv_java480.dll to the JDK bin folder. On my computer, my JDK is at: C:\Program Files\Eclipse Adoptium\jdk-21.0.1.12-hotspot\bin

Step 3: Put the jar file onto the system environment path. On windows: under System variables, look for a variable named CLASSPATH: if it exists, select it and click Edit, and add the following into the path: .;D:\opencv\build\java\opencv-480.jar

Note: if you would like to use some advanced face recognition methods like Eigenface, Fisherfaces, LBPH face recognition methods, the opencv contrib module should be installed and you need to build it by yourself.

MacOS:

For Mac user, you need to build by yourself.

Assuming you have a JDK installed and you've set JAVA_HOME

Installing homebrew (package manager for linux/ macOs)

Step 1:

Download homebrew by running:

/bin/bash -c "\$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)" in your terminal

Step 2:

Add path to homebrew, in the terminal:

nano ~/.zshrc

Append <export PATH="/opt/homebrew/bin:\$PATH"> to file

Save and exit (Ctrl X + y + enter)

Step 3:

Reload shell for changes to take place

source ~/.zshrc

Installing Apache Ant (A Java-based build tool) + CMake (Cross-platform build system generator)

Step 4: Install ant and make

```
brew install cmake ant
```

Verify that it is installed:

```
cmake --version
```

```
ant -version
```

Installation of OpenCV

For OpenCV installation for macOS, we need to build and compile from the source code so we need ant and cmake.

Step 5: Download opencv source from <https://opencv.org/releases/> (opencv4.8.0)

Step 6: Unzip download and create a build folder

(Assuming its unzipped in ~/Downloads)

run:

```
cd ~/Downloads/opencv-4.8.0
```

```
mkdir build && cd build
```

Step 7: Run CMake command to configure the build for OpenCV, setting it to a release version with Java support and telling it to build

```
cmake -DCMAKE_BUILD_TYPE=Release \  
-DCMAKE_INSTALL_PREFIX=/usr/local/opencv \  
-DBUILD_opencv_java=ON \  
-DBUILD_EXAMPLES=ON \  
-DINSTALL_C_EXAMPLES=OFF \  
-DINSTALL_PYTHON_EXAMPLES=OFF \  
..
```

Step 8: Compile OpenCV

```
make -j$(sysctl -n hw.ncpu)
```

`$(sysctl -n hw.ncpu) **` this just tells the device to use all cores for compiling

Step 9: Put the jar file onto the system environment path, due to the prefix in make command, opencv would be found in `/usr/local/opencv`

`nano ~/.zshrc`

Append `<export CLASSPATH=$CLASSPATH:/usr/local/opencv/share/java/opencv4/opencv-480.jar:>`

(Or change the path to wherever your .jar file is located)

Step 10: Copy `libopencv_java480.dylib` to the jdk bin folder

`sudo cp /usr/local/opencv/share/java/opencv4/libopencv_java480.dylib $JAVA_HOME/lib`

(Or change the path to wherever the `libopencv_java_480.dylib` file is located)

Step 11:

Finally, you can verify that if you have installed opencv successfully by the following codes:

```
import org.opencv.core.Core;

public class OpenCVTest {
    public static void main(String[] args) {
        System.loadLibrary(Core.NATIVE_LIBRARY_NAME);
        System.out.println("OpenCV version: " + Core.VERSION);
    }
}
```

`javac -cp $CLASSPATH OpenCVTest.java`

`java OpenCVTest`

OR, you can add the referenced library in `settings.json` of vscode so that you don't have to specify the class path every time you want to compile

In `.vscode/settings.json`, add:

```
{
  "java.project.referencedLibraries": [
    "/usr/local/opencv/share/java/opencv4/opencv-480.jar"
  ]
}
```

]

}

*or where ever your opencv jar file is found

Helpful commands to locate path of a file if you're unable to

find ~/<Folder you want to search in> -name "File you want to find>"

e.g. find ~/Downloads -name "libopencv_java*.dylib"